

B13705023 蔡承勸 HW6

1. Short Answer

1.

(a)

- Static Web Server 表現出的 web page 是以 HTML, JS, CSS 等寫的。當伺服器收到請求，就會回傳 response 回去，不加上任何額外過程(很簡單的一對一 HTTP Request -> HTTP Response 過程)。頁面會保持一樣直到有人手動更改，而且沒有資料庫。
- Dynamic Web Server 表現出的 web page 則可能是由 CGI, AJAX, ASP 等寫的。對於不同的瀏覽者可能會有不同的資訊，且會花相較 Static 更多時間載入。當伺服器收到請求，會先呼叫一個 application program，而這個程式執行後會得到 HTML 資訊，再以 HTTP response 回傳。畫面上會如名字一樣動態改變，像是股價，天氣資訊等等。相較靜態的，會使用到資料庫。

參考資料：

<https://www.geeksforgeeks.org/computer-networks/difference-between-static-and-dynamic-web-pages/>

(b)

- Forward Proxy: 為一個存在於用戶和伺服器之間的正向代理伺服器，能夠：
 - 突破因距離太遠的溝通限制
 - 隱藏真實IP (客戶端)，伺服器看到的 ip 並不是 client 的，而是 proxy 的
 - 提升訪問速度 (因為有緩存機制)
- Reverse Proxy: 也是一個存在於用戶和伺服器之間的反向代理伺服器，類似於 forward proxy，也能夠：
 - 隱藏真實IP (伺服器)，用戶看到的 ip 並不是 server 的，而是 proxy 的
 - Load Balancing，能夠根據全部伺服器的負載狀況，去分配流量
 - 可以做為應用層的 firewall，防 ddos

參考資料：

<https://ithelp.ithome.com.tw/articles/10313671>

(c)

- Load Balancing: 即為把網路流量在多個伺服器中有效率地分配，以最佳化應用程式的 availability 和保證用戶可以有流暢的體驗。
- 舉例來說，Round Robin 是一個方式，當後端伺服器都很相似的時候，讓 load balancer (可以是 proxy)，去指定流量怎麼走，使得伺服器們輪流去接收一個個請求。

參考資料：

<https://www.ibm.com/think/topics/load-balancing#:~:text=Load balancing is the process of distributing network,application availability and ensure a positive end-user experience.>

<https://freeloadbalancer.com/blog/round-robin-load-balancing-explained>

2.

(a)

- Nginx's Event-driven architecture: 每個工作進程都是單執行緒的，並且能夠透過高效的事件通知機制 (如 epoll on Linux) 同時處理成千上萬個連線。
當一個請求進來時，工作進程不會等他(例如等待磁碟 I/O 或後端回應)，而是將這個請求註冊一個事件(即為上述的 event-driven)，然後立即去處理下一個請求。當之前的 I/O 操作完成後，事件機制會通知工作進程，工作進程再回過頭來處理這個請求的後續步驟。
- Apache's Process-driven architecture: 是最古老且最穩定的模式，伺服器會預先 fork 數個 child processes。每個子進程在同一時間只能處理一個請求。當一個請求進來時，會被分配給一個閒置的子進程，直到請求處理完畢。

參考資料：

<https://medium.com/@daniel.doody/web-servers-a-high-level-overview-of-nginx-vs-apache-ba87f923c024>

<https://ithelp.ithome.com.tw/articles/10384470>

(b)

- Master process: 只有一個，除了管理 worker processes 之外也要負責讀取 nginx 的配置文件 (nginx.conf)，根據配置啟動相應的 worker process，也要處理信號，像是 nginx -s reload, nginx -s stop 等。
- Worker process: 處理真正的請求，接收和處理 http 連線，把資料寫到 sockets，處理快取，php-fpm 等。

參考資料：

<https://www.cnblogs.com/jianmingyuan/p/13960906.html>

<https://www.server.hk/blog/51035/>

(c)

- User of master process: root，因為要 listen on ports
- User of worker process: 非 root，無權限者 (為實現 principle of least privilege)，若被攻擊那攻擊者不會直接取得 root 權限。

參考資料：

chatGPT

3.

(a)

- PKI (Public Key Infrastructure) 為處理數位簽章和加密金鑰的 framework。它支持非對稱加密，也就是有兩副加密金鑰(private public) 用來加密和解密資訊。PKI 包含:
 - CA: 核發數位簽章的機構
 - RA: 作為核發簽章之前的審核者
 - 數位簽章: 數位 id 來連結公鑰到認證過的單位
- TLS 需要 PKI 來實施一個握手。當我們現在連接到一個網站上時，伺服器會給我們的 browser 看一個已信任的 CA 核發的數位簽章，其中包含伺服器的公鑰。接下來，我們的瀏覽器會檢查:
 - 簽章是被信任的 CA 簽過的嗎？
 - 是合格的而且還沒過期嗎？

◦ 有被撤銷嗎？

- 如果以上都通關了，那一個加密過的連線就會成功建立，具體來說是使用伺服器的公鑰加密瀏覽器生成的 session key，然後送過去。接著伺服器會用他的私鑰解鎖 session key，最後兩方都會用這個 shared session key 來保證快速而且加密的連線。

參考資料：

<https://emudhra.com/en-us/blog/introduction-to-transport-layer-security-tls-public-key-infrastructure-pki#:~:text=If TLS is the secure envelope%2C PKI is,framework that manages digital certificates and encryption keys.>

(b)

- ACME(Automatic Certificate Management Environment) 協定是目的為自動化頒發和更新數位憑證流程的開放標準，ACME 的開發旨在簡化整個流程，並已被許多憑證授權單位 (CA) 廣泛採用，並已成為網路標準 (RFC 8555)。
- ACME 透過定義 Web server 和 CA 之間的通訊標準協定來自動化整個流程。Web 伺服器 (ACME 用戶端) 向 CA (ACME 伺服器) 傳送請求，以取得一個或多個網域的憑證。然後，CA 會要求客戶端證明其對網域的所有權 (稱為 challenges)，通常是透過在 Web 伺服器上放置特定文件來實現的。一旦 CA 驗證完成，客戶就會向 CA 發送一個 CSR (Certificate Signing Request)，接著 CA 就會向客戶端頒發證書，客戶端會自動安裝該證書。該過程可以完全自動化，從而可以輕鬆實現初始設定和無縫續訂。

參考資料：

https://en.wikipedia.org/wiki/Automatic_Certificate_Management_Environment
<https://www.ssl.com/zh-TW/文章/什麼是-Acme-協議/>

(c)

- 用網域名稱發放的好處：CA 們可以透過 DNS-01, HTTP-01 挑戰來很方便驗證域名的所有權，算是一個標準的作法。而且若用 DN 來驗證也不會把自己的 IP 暴露在光天化日之下。除此之外，域名是可以移動的，表示如果換了 IP 但域名沒換，那憑證還是可以使用。

- 用 IP 地址發放的壞處：如果今天是 Private IP 那完全不用說，根本驗證不了所有權；如果今天是 Public IP 的話，甚至會需要區域網際網路註冊管理機構 (RIR) 的證明，這通常只有大公司或是 isp 可以提供。更不好的還有許多瀏覽器根本不信任給 IP 地址的憑證。

參考資料：

<https://stackoverflow.com/questions/2043617/is-it-possible-to-have-ssl-certificate-for-ip-address-not-domain-name>

(d)

- SSL Termination: 算是一種優化策略 (避免消耗太多解密加密資料的 cpu 用量)，就是會在 reverse proxy 等加密資料的終點解密 TLS/SSL 加密的流量，在解密之後，就會以明文在後端資料庫移動了。
- Nginx 在 ssl termination 之後會加入 "proxy headers" 這種東西，會把 \$scheme 標明 https，以讓後端伺服器能夠意識到這是 https 請求。

參考資料：

https://blog.csdn.net/weixin_41737291/article/details/149158499

<https://www.ssldragon.com/blog/what-is-ssl-termination/>

chatgpt

2. Web Server Configurations

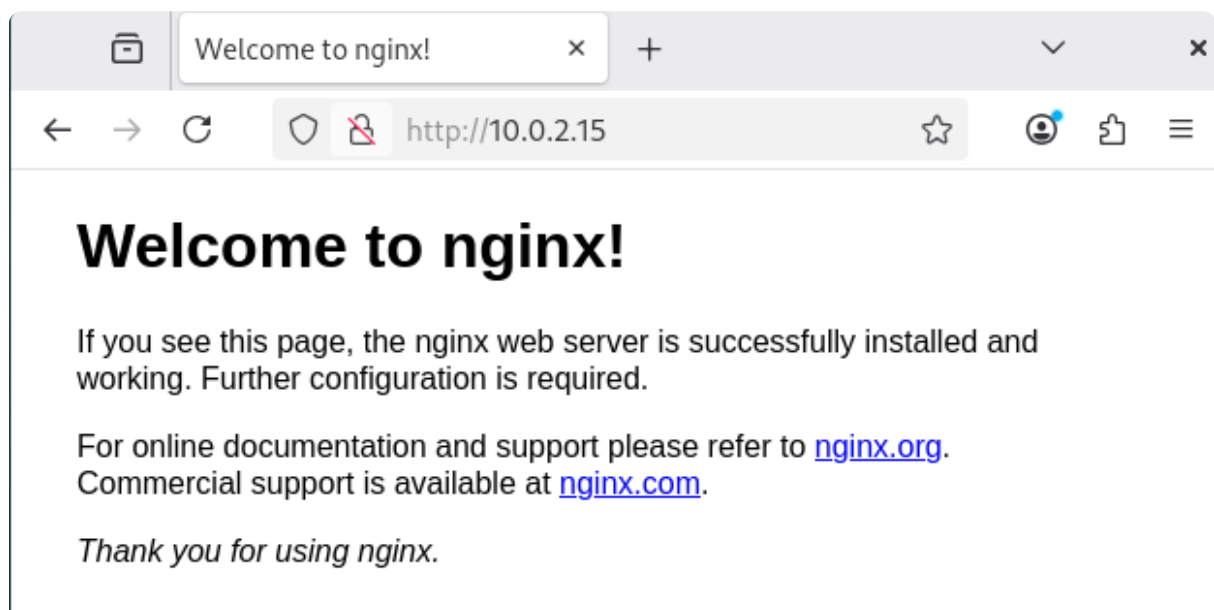
1.

- `sudo apt update && upgrade -y` 確保最新
- `sudo apt install -y nginx ufw` 把 nginx 和 ufw 都載下來
- `sudo systemctl enable --now nginx`
- `sudo ufw default deny incoming` 拒絕所有進入連線
- `sudo ufw default allow outgoing` 允許所有出去連線
- `sudo apt install -y openssh-server`
- `sudo ufw allow OpenSSH` allow 22/tcp
- `sudo ufw allow 'Nginx HTTP'` allow 80
- `sudo ufw enable`

- `ufw status verbose` 檢查防火牆

```
root@nasa:/home/vboxuser# ufw status verbose
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), disabled (routed)
New profiles: skip
```

- | To | Action | From |
|--------------------------|----------|---------------|
| -- | ----- | ---- |
| 22/tcp (OpenSSH) | ALLOW IN | Anywhere |
| 80/tcp (Nginx HTTP) | ALLOW IN | Anywhere |
| 22/tcp (OpenSSH (v6)) | ALLOW IN | Anywhere (v6) |
| 80/tcp (Nginx HTTP (v6)) | ALLOW IN | Anywhere (v6) |



參考資料：

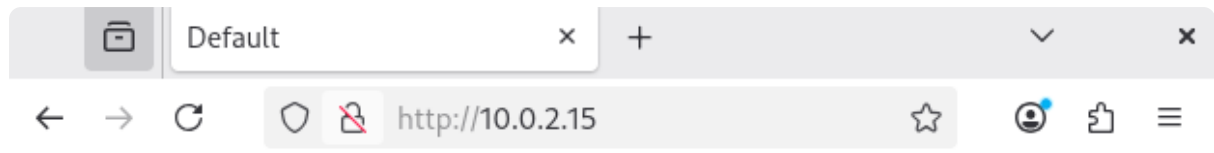
<https://www.digitalocean.com/community/tutorials/how-to-set-up-a-firewall-with-ufw-on-ubuntu>

2.

- 在 `/var/www/html` 下新增 `index.html`

```
<!DOCTYPE html>
<html>
  <head><title>Default Page</title></head>
  <body>
    <h1>b13705023: Default</h1>
  </body>
</html>
```

- 把原本的 index.nginx-debian.html 刪了



• **b13705023: Default**

參考資料：

<https://futurestud.io/tutorials/nginx-how-to-serve-a-static-html-page#:~:text=This tutorial shows you how to serve a,you through the required paths%2C files%2C and configurations.>

3.

- `sudo mkdir /var/www/html/forbidden`
- `in /var/www/html nano 403.html`

```
<!DOCTYPE html>
<html>
  <head><title>403 Forbidden</title></head>
  <body>
    <h1>b13705023: 403 Forbidden</h1>
  </body>
</html>
```

- `sudo nano /etc/nginx/sites-available/default` 去這個設定檔
- 在 location / 的 block 之下加入

```
location /forbidden {
    deny all;
    error_page 403 /403.html;
}
```

- `sudo nginx -t` 檢查配置
- `sudo systemctl reload nginx`



```
root@nasa:/var/www/html# curl -I http://10.0.2.15/forbidden
HTTP/1.1 403 Forbidden
Server: nginx/1.22.1
Date: Fri, 17 Oct 2025 04:38:44 GMT
Content-Type: text/html
Content-Length: 132
Connection: keep-alive
ETag: "68f1c3b1-84"
```

參考資料：

https://nginx.org/en/docs/beginners_guide.html

4.

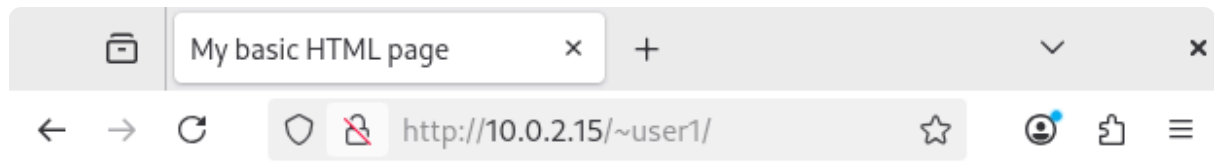
- `sudo nano /etc/nginx/sites-available/default` 修改 default conf
- 在 `server_name _;` 底下加入
- ```
location ~ ^/~(.+?)(/.*)?$ {
 alias /home/$1/htdocs$2;
 index index.html index.htm;
 autoindex on;
}
```
- `sudo nginx -t && sudo systemctl restart nginx`
- `sudo mkdir -p /etc/skel/htdocs`
- `sudo adduser --disabled-password --gecos "" user1` 創建一個測試用 user1，而在創建的同時，`/etc/user1/htdocs` 會被自動建立
- 所以我們現在進去 `/etc/user1/htdocs` 加入 `index.html`，裡面寫入



```
<title>My basic HTML page</title>
<body>
<p style="width: 100%; font-weight: bold; font-size: 60px; text-align: center;"
user1: UserDir
</p>
</body>
</html>
```



- `sudo chmod 755 /home`
- `sudo chmod 755 /home/user1`
- `sudo chown -R user1:user1 /home/user1`
- `sudo chmod -R 755 /home/user1/htdocs`
- 以上是確保 nginx 可以 traverse /home 然後讀 htdocs，還有確保 user 自己有權限
- `sudo systemctl reload nginx`



# user1: UserDir

•

參考資料：

<https://geekrewind.com/enable-nginx-userdir-on-ubuntu-24-04/>

## 5.

- `sudo mkdir -p /opt/serverA && sudo nano /opt/serverA/index.html`
- 在裡面寫入 I am Server A
- 對 server B 做一模一樣的事，換成 serverB 就好
- `cd /opt/serverA`

- `python3 -m http.server 7777` 啟動 python serverA listening on 7777
- `cd /opt/serverB`
- `python3 -m http.server 8888` 啟動 python serverB listening on 8888
- `sudo nano /etc/nginx/sites-available/reverse-proxy`
- 加入以下

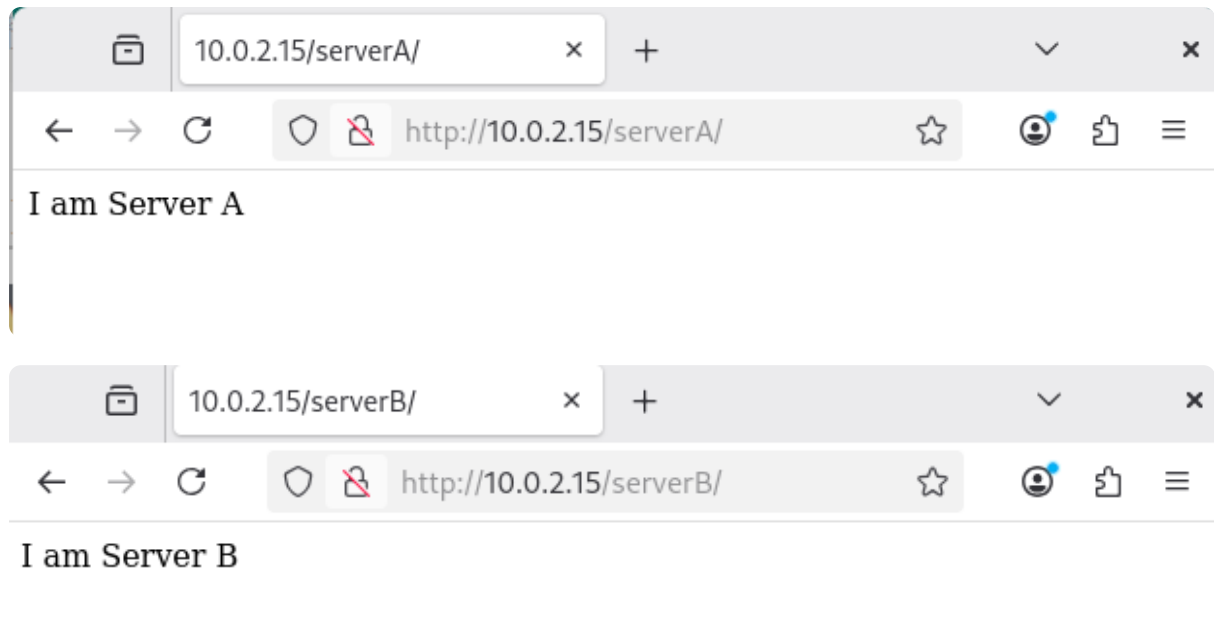
```
server {
 listen 80 default server;

 location = /serverA { return 301 /serverA/; }
 location = /serverB { return 301 /serverB/; }

 # proxy /serverA/ to backend at port 7777
 location /serverA/ {
 proxy_pass http://127.0.0.1:7777/;
 proxy_set_header Host $host;
 proxy_set_header X-Real-IP $remote_addr;
 proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
 proxy_set_header X-Forwarded-Proto $scheme;
 proxy_pass_request_headers on;
 }

 # proxy /serverB/ to backend at port 8888
 location /serverB/ {
 proxy_pass http://127.0.0.1:8888/;
 proxy_set_header Host $host;
 proxy_set_header X-Real-IP $remote_addr;
 proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
 proxy_set_header X-Forwarded-Proto $scheme;
 proxy_pass_request_headers on;
 }
}
```

- 上面的 default server 很重要！要把他從 default 的 config 拉到這裡，不然會一直顯示 404 not found
- `sudo ln -sf /etc/nginx/sites-available/reverse-proxy /etc/nginx/sites-enabled/reverse-proxy` symbolic link 過去到 enabled
- `sudo nginx -t && systemctl reload nginx`



參考資料：

[https://nginx.org/en/docs/http/server\\_names.html](https://nginx.org/en/docs/http/server_names.html)

<https://ithelp.ithome.com.tw/articles/10394760>

<https://go.lightnode.com/tech/how-to-configure-nginx-reverse-proxy>

## 6.

- `sudo mkdir -p /opt/serverC && sudo nano /opt/serverC/index.html`
- `cd /opt/serverC`
- `python3 -m http.server 9999` 啟動 python serverC listening on 9999
- `sudo nano /etc/nginx/sites-available/reverse-proxy`
- 在 server 之外加入

```
upstream lb_pool {
 # round robin default among these servers
 server 127.0.0.1:7777 max_fails=3 fail_timeout=30s;
 server 127.0.0.1:8888 max_fails=3 fail_timeout=30s;
 # backup server used only when others are down
 server 127.0.0.1:9999 backup;
}
```

- 在 server 裡面加入一個 location

```
location = /balance { return 301 /balance/; }
```

```
location /balance/ {
 proxy_pass http://lb_pool/;
 proxy_set_header Host $host;
 proxy_set_header X-Real-IP $remote_addr;
 proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
 proxy_set_header X-Forwarded-Proto $scheme;
 proxy_set_header X-Forwarded-Host $host; proxy_pass_request_headers on;
 proxy_buffering off;
}
```

- `sudo ln -sf /etc/nginx/sites-available/reverse-proxy /etc/nginx/sites-enabled/reverse-proxy` symbolic link 過去到 enabled
- `sudo nginx -t && systemctl reload nginx`

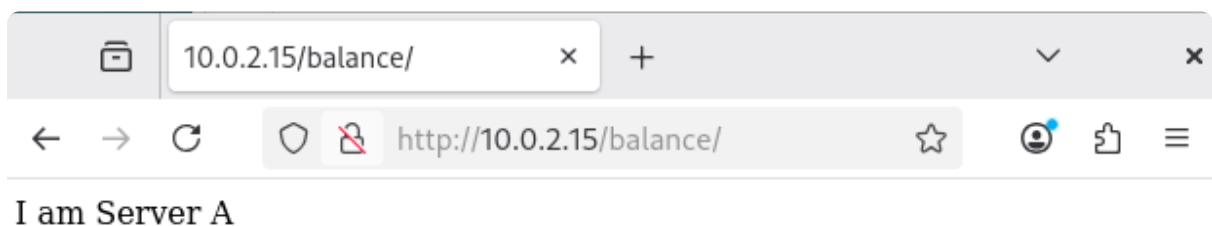
```
root@nasa:/etc/nginx/sites-enabled# for i in 1 2 3 4 5; do curl -s http://127.0.0.1/balance/ -o - | sed -n '1p'; echo; sleep 0.2; done
I am Server A

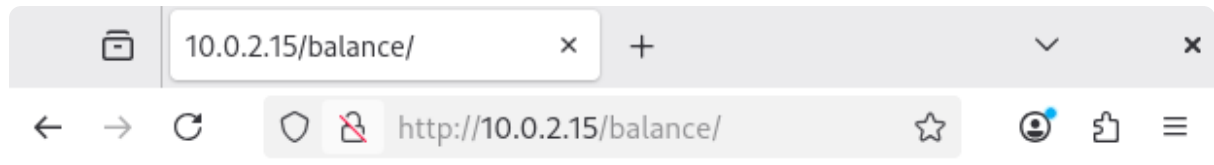
I am Server B
• I am Server A

I am Server B

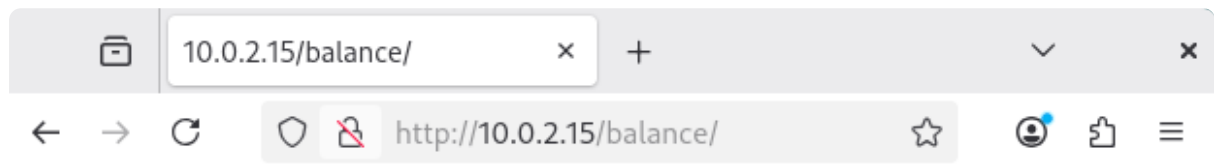
I am Server A
```

展現 round robin load balancing





- I am Server B



- I am Server C

把 A B Server 都關掉後

```
root@nasa:/etc/nginx/sites-enabled# for i in 1 2 3 4 5; do curl -s http://127.0.0.1/balance/ -o - | sed -n '1p'; echo; sleep 0.2; done
I am Server C
I am Server C
• I am Server C
I am Server C
I am Server C
```

參考資料：

[https://nginx.org/en/docs/http/load\\_balancing.html](https://nginx.org/en/docs/http/load_balancing.html)

chatGPT